Phase 3 Testing Report — Turtle Escape

CMPT 276 — Group 11

1. Unit Tests

For the testing, we wanted to isolate individual features and check that each one worked on its own before worrying about how they interacted. The first thing that we had to figure out was how to actually run tests against our game, seeing as GamePanel loads images, sounds, and map files in its constructor and immediately tries to set up Swing. Eventually, we ended up building a class called FakeGamePanel that uses sun.misc.Unsafe to allocate GamePanel without calling its constructor at all. This allows us to write tests against the real production classes without any of the resource loading getting in the way.

1.1 Node (NodeTest — 6 tests)

Node is the class that the A* pathfinding algorithm uses to represent each tile on the map. It stores gCost, hCost, fCost, some boolean flags (open, checked, solid), and a parent reference for reconstructing the path once the goal is found. For this test, we verified that the col and row passed to the constructor are stored correctly and that all cost fields start at zero. Lastly, we also checked that all flags default to false, and that the parent reference starts null.

1.2 KeyHandler (KeyHandlerTest — 13 tests)

KeyHandler tracks which movement keys are currently held down, as we wanted to make sure every key correctly sets and clears its flag, as a bug here would break player movement. We created fake KeyEvent objects using a dummy AWT Component and fired them through the real keyPressed and keyReleased methods. Tests cover all five keys used in the game (W/A/S/D/ESC). For both the press and release checks, we check that all flags start false, verify that pressing two keys at once sets both flags independently, and confirm that releasing one key doesn't accidentally clear a key that's still held.

1.3 Player (PlayerTest — 11 tests)

The Player class handles score changes when items are collected and death triggering when an enemy is contacted. We tested that picking up each item type (Seaweed, Jellyfish, Shrimp, IceCream) adds the right score value and removes the object from the

array. In addition, we also check that multiple pickups accumulate correctly, and that passing index 999 (meaning no object was touched) does nothing. For contactEnemy, we checked that a valid enemy index with invincible set to false transitions the game to DEAD, as well as making sure that index 999 doesn't trigger death, and that an invincible player survives the contact.

1.4 CollisionChecker (CollisionCheckerTest — 8 tests)

In CollisionChecker alot of the core gameplay mechanics revolve around this class, so its test results mattered most to us. For checkPlayer, we tested that it returns true when an enemy is placed directly on top of the player and false when the enemy is far away. We also checked that it was false when the enemy is just one pixel outside the hitbox, and that it doesn't crash when gp.player is null. For checkEntity, we checked overlap detection, no-overlap returning 999, null slots being skipped, and an entity not detecting itself as a collision target.

1.5 Pathfinder (PathfinderTest — 12 tests)

Pathfinder runs the A* search that enemies use to chase the player. We tested that instantiateNodes creates a full grid with correct positions, and then also tested resetNodes to check that it clears all flags, empties the open/path lists, and resets the counters. We also checked that getCost correctly computes fCost as gCost + hCost with the start node having gCost zero and the goal node having hCost zero. Additionally, we checked that search finds a path on a clear map, and that the last node in pathList is the goal, and that search correctly returns false when the goal is surrounded by solid tiles on all four sides.

2. Integration Tests

Once we were confident the individual pieces worked in isolation, we moved on to testing how they interact with each other.

2.1 Enemy-Player Contact (EnemyPlayerContactTest — 9 tests)

This tests the full coding stream from an enemy's position through CollisionChecker.checkPlayer() and Entity.checkCollision() all the way to GamePanel.GameState.DEAD. We tested all four directions of approach to make sure

the death state triggers regardless of the way in which the enemy is coming from. We also tested that a far-away enemy doesn't kill the player, and that a non-enemy entity (type 0) touching the player doesn't trigger death, and that our passive overlap check correctly detects contact even when neither entity is moving.

2.2 Tile Collision (TileCollisionTest — 12 tests)

This tests the interaction between CollisionChecker.checkTile() and the tile grid in TileManager. We set the individual tiles to solid or walkable and checked whether collisionOn gets set correctly for movement in either direction. We also added four boundary tests to make sure entities at the edges of the map don't throw ArrayIndexOutOfBoundsException. This was because during phase 2 we encountered a bug when running the game with larger enemy hitboxes.

2.3 Player Object Pickup (PlayerObjectPickupTest — 7 tests)

This tests the interaction between CollisionChecker.checkObject() and Player.pickUpObject(). We verified that checkObject returns the correct index when an object overlaps the player and 999 when nothing is in range, that the full pickup pipeline correctly updates the score and removes the object for Seaweed and IceCream, that two sequential pickups accumulate the score correctly, that an object the player isn't overlapping doesn't get picked up, and that null slots in the object array don't cause a crash.

3. Code Coverage

Package	Line Cov.	Branch Cov.	Missed Lines	Missed Branches
input (KeyHandler) ai	98%	100%	1 / 26	0 / 14
(Pathfinder, Node)	95%	81%	3 / 106	9 / 38
collision (CollisionChecker)	82%	47%	7 / 119	40 / 65
entity (Entity, Player)	15%	10%	217 / 266	100 / 114
enemy (SuperEnemy)	0%	0%	76 / 76	34 / 34
tile (TileManager)	0%	0%	112 / 114	37 / 38
UI (GamePanel, UI)	3%	0%	302 / 316	82 / 86
object	2%	5%	197 / 207	44 / 46
Total	**36% (avg)**	**30% (avg)**	**915 / 1230**	**346 / 435**

We measured coverage using JaCoCo. To set it up, we moved the plugin from pluginManagement into the active plugins section of pom.xml. Then, we ran mvn clean test, which automatically generates the report. At the end, we got 26% line coverage and 23% branch coverage across all 28 of the classes.

3.1 Packages with Good Coverage

The packages that contain the core logic ofthe game came out well. KeyHandler hit 98% line and 100% branch coverage, whichmakes sense as it's just flag setting with no dependencies. Pathfinder and Node togetherhit 95% line and 81% branch. CollisionChecker hit 82% line coverage and47% branch coverage, and the uncovered branches are mostly the overlay tile checksand some of the entity/object collision paths that are harder to hit in isolation.

3.2 Packages with Low Coverage

Several of our packages are near zero, but this is expected given how the game is structured, rather than a gap in our testing effort.

• UI and GamePanel (3% line, 0% branch): These classes need a live Graphics2D context and a Swing JPanel to do anything useful. You can't render a death screen or draw tiles without a display, so these simply can't be tested in an environment without a GUI.

• TileManager (0% line, 0% branch): TileManager reads map files and loads PNG images in its constructor. Our FakeGamePanel removes this entirely rather than trying to instantiate the real one.

• SuperEnemy (0% line, 0% branch): SuperEnemy loads sprite images in its constructor via setUp(), which requires PNG files on the classpath at test time. Without them, the constructor crashes, subsequently meaning we couldn't instantiate it.

• Object package (2% line, 5% branch): This is the same problem as SuperEnemy, as the object classes load images in their constructors. The small amount of coverage that does appear is because the SuperObject base class is being used through anonymous subclasses in our test helpers.

• Entity and Player (15% line, 10% branch): These are partially covered through PlayerTest and the integration tests. However, a lot of the branches, like animation frame toggling, diagonal movement speed, the sound tile switching, and the win condition check require a running game loop and real tile data that we can't set up in isolation.

4. Test Quality

A few things we tried to be consistent about across all the tests:

• Test method names describe exactly what's being checked and what the expected result is, so if/when something fails, we'll know immediately what broke without having to read the test body.

• Each test focuses on one behaviour. For example, we assert multiple things in one test, like checking both the score and that the object was removed after a pickup. These are logically the same action, so it makes sense to keep them together.

• Every assertTrue and assertFalse has a message explaining what went wrong if it fails.

• Every test class uses @BeforeEach to build a fresh FakeGamePanel so no state leaks between tests.

• We tested edge cases that could've caused errors like index 999, null array entries, entities colliding with themselves, invincibility, and positions at map boundaries.

5. Findings

5.1 Bugs Found and Fixed

Writing the tests brought a few issues to our attention that we were able to fix. The first bug was a hitbox positioning issue with the player. The solidArea was originally Rectangle(8, 16, 32, 32), which pushed the collision box down by 16 pixels, forcing it to star halfway down the sprite. Enemies could then overlap the top half of the player visually without triggering contact, which made the collision fault. We adjusted it to Rectangle(4, 4, 40, 40) to cover most of the sprite evenly, and it now works.

5.2 Challenges We Ran Into

The biggest challenge by far was figuring out how to test anything at all, given how GamePanel is structured. Its constructor does a large amount of work, as it loads the

tile map from a text file, loads 50+ PNG images for tiles and sprites, opens WAV files for all the sound effects, creates a Pathfinder that allocates a 100x22 node grid, and registers Swing listeners. Unfortunately none of these tests work in an environment without the use of a GUI. Therefore, we ended up using sun.misc.Unsafe to allocate instances without calling constructors and then wiring all the fields manually via reflection.

Another thing that took some time to figure out was that maxWorldCol and maxWorldRow in GamePanel are declared as public final int maxWorldCol = 100. This is because they have literal constant initialisers, so the Java compiler inlines those values everywhere they're used at compile time. Consequently, even if you write a different value to the field via reflection, the Pathfinder loop and CollisionChecker array accesses still use 100 and 22, meaning they don't read the field at runtime. After realising this, we were able to fix it by allocating the tile arrays to the real 100x22 size rather than trying to override the constants.

Sound also proved itself to be quite tricky, as allocating Sound with Unsafe left the soundURL array as null, so the first time pickUpObject called playSE(), it crashed inside setFile() trying to index into a null array. We overcame this eventually by writing a NoOpSound subclass that overrides every method to do nothing, so the Sound constructor runs normally (initialising the URL array) but no actual audio system code ever fires during tests.

5.3 Code Improvements from Testing

Beyond the bug fixes, going through the testing process pushed us to clean up a few things. The checkTile method had four copies of the same Math.max(min, Math.min(max, value)) expression, after noticing that we pulled it into a private clamp() helper. We also separated the passive contact check from the directional prediction check, which made the distinction between movement blocking and damage detection clearer in the code. the FakeGamePanel class also turned out to be a solid piece of reusable infrastructure that any future tests we may choose to do on this project can use without having to re-implement the Unsafe allocation logic from scratch.